Compiler Principles (Honor Track)

timetraveler314
University of Genshin
timetraveler314@outlook.com

1. Lexical Analysis

1.1. Converting Regular Expressions to DFA: Taking Derivatives

> **Definition 1.1.** Derivative of a Language
>
> Given a language L and a symbol a, the derivative of L with respect to a, denoted as $D_a(L)$, is the set of strings that can be obtained by removing the prefix a from each string in L.

Consider the problem of whether a string s is in the language L. We see that this can be solved step by step by taking the derivative of r with respect to each symbol in s. If the final derivative has ε in it, then s is in L; otherwise, s is not in L.

Now we will show a theorem on derivative of regular languages, by whose construction we can convert a regular expression to a DFA.

> **Theorem 1.1.** Derivative of Regular Languages
>
> Given a regular expression r and a symbol a, the derivative of r with respect to a, denoted as $D_a(r)$, is a regular expression such that $L(D_a(r)) = D_a(L(r))$.

Proof. We will prove this theorem by induction on the structure of r.

- $D_a(\varepsilon) = \varnothing, D_a(a) = \varepsilon, D_a(b) = \varnothing$ for $b \neq a$.
- $D_a(r \mid s) = D_a(r) \mid D_a(s)$,
- $D_a(rs) = D_a(r)s \mid \mathrm{emp}(r)D_a(s)$, where $\mathrm{emp}(r)$ is ε if r can match the empty string, and $\varnothing$ otherwise. Intuitively, if r can match the empty string, then we can either remove the prefix a from s.
- $D_a(r^*) = D_a(r)r^*$.

The above induction uses the new operation emp, which can be defined as follows:

- $\mathrm{emp}(\varepsilon) = \varepsilon, \mathrm{emp}(a) = \varnothing$ for $a \neq \varepsilon$,
- $\mathrm{emp}(r \mid s) = \mathrm{emp}(r) \mid \mathrm{emp}(s)$,
- $\mathrm{emp}(rs) = \mathrm{emp}(r)\,\mathrm{emp}(s)$,
- $\mathrm{emp}(r^*) = \varepsilon$.

□

As for regular expressions, taking derivatives is just like performing a step of the DFA state transition. $D_a(r) = s$ stands for that if we are in state r and we read symbol a, we will go to state s. When the result becomes ε, we can say that the string is in the language. Hence by iteratively taking derivatives, we can build all the states (regular expressions) of the DFA, until no more new states can be generated.

2. Syntax Analysis

2.1. Grammar, Derivation and Reduction

> **Definition 2.1.** Derivation and Reduction
>
> Given a grammar $G = (V, T, P, S)$, if $\alpha \to \beta$ is a production rule in P, and γ, δ are strings consisted of $V \cup T$, then we say $\boldsymbol{\gamma\alpha\delta}$ **derives** $\boldsymbol{\gamma\beta\delta}$ **in one step**, denoted as $\gamma\alpha\delta \Rightarrow \gamma\beta\delta$.
>
> **Derivation** $\alpha_0 \overset{*}{\Rightarrow} \alpha_n$ is a sequence of derivation steps, and **reduction** (e.g. $\gamma\alpha\delta \Leftarrow \gamma\beta\delta$) is the reverse process of derivation.

2.2. Ambiguity

> **Definition 2.2.** Ambiguity
>
> A grammar is ambiguous if there exists a string that has more than one leftmost derivation.

> **Note.**
>
> Ambiguity has more equivalent definitions, such as **rightmost derivation**, **parse tree**. To see the equivalence, we can consider the trivial bijection between left(right)most derivation and parse tree.

2.2.1. Examples of Unambiguous Grammar

By the key observation of Note, we can see that grammars that are $\mathrm{LL}(k)$ or $\mathrm{LR}(k)$ are unambiguous, since their corresponding parsers uniquely determine the leftmost or rightmost derivation.

> **Example 2.1.**
>
> Consider the following grammar:
>
> $$S ::= SS+ \mid SS* \mid \texttt{a}.$$
>
> This grammar is $\mathrm{LR}(0)$, and hence unambiguous.

2.2.2. Eliminating Ambiguity

In cases like the grammer of `Expr`, we can eliminate ambiguity by adding more non-terminals and productions. ...

2.3. Top-Down Parsing and Recursive Descent Parsing

In this section, we introduce the syntax $w : \beta$ to denote that w is a string of terminals that can be derived from β, where β is the target of the parsing.

> **Definition 2.3.** Top-Down Parsing
>
> Top-down parsing is a parsing strategy that starts from the root of the parse tree and works its way down to the leaves. It is also called **predictive parsing**.

Specifically, a simple algorithm can be given as follows:

1. We process the input string from left to right.
2. At first, we have the relation string : S, e.g. $[[][]] : S$.

3. From the **leftmost** non-terminal in β, we choose a production rule to replace it.
4. If a terminal occurs in the leftmost position of β, we match it with the input string.
5. Repeat the above steps until the input string is empty.

However, the algorithm above is not always applicable. When there are multiple rules for the same non-terminal, we need to choose the right one. This is the so-called **backtracking** problem. The solution is either to try multiple rules by backtracking, or to use a **lookahead** to predict the next rule.

2.3.1. Predictive Parsing and Lookahead

The intuition here is that we can sometimes predict the next rule by looking at the next few symbols in the input string. Is this possible for every grammar? The answer is no. Several situations can cause the failure of predictive parsing. However, some of them can be transformed into a form that can be parsed predictively.

- **Left Recursion**:

– Direct left recursion $A \to A\alpha \mid \beta$ can be transformed into $A \to \beta A'$, $A' \to \alpha A' \mid \varepsilon$;

– There is also indirect left recursion $A \overset{+}{\Rightarrow} A\alpha$. For example, the grammar

$$S ::= A\texttt{a} \mid \texttt{b}$$
$$A ::= S\texttt{d} \mid \varepsilon.$$

To eliminate indirect left recursion, we may rewrite the grammar into a *strictly descending* form. Formally, we arrange the non-terminals in a sequence $A_1, A_2, ..., A_n$ such that latter non-terminals do not left-recursively derive former non-terminals (i.e. no $A_i ::= A_j\gamma$ for $i > j$). Traverse A_i sequentially and replace the left recursion with the following rules:

1. For every rule $A_i ::= A_j\gamma (j < i)$, replace it with $A_i ::= \delta_1\gamma \mid ... \mid \delta_k\gamma$, where $A_j ::= \delta_1 \mid ... \mid \delta_k$.
2. Further eliminate the direct left recursion in A_i.

- **Left Factoring**: When $A ::= \beta_1 \mid \beta_2$ s.t. $\mathrm{FIRST}(\beta_1) \cap \mathrm{FIRST}(\beta_2) \neq \varnothing$, predictively parsing A is impossible. However, if there are multiple rules starting with the same prefix, we can factor out the common prefix.

$$A ::= \alpha\beta_1 \mid \alpha\beta_2 \Longrightarrow A ::= \alpha A', A' ::= \beta_1 \mid \beta_2.$$

Now we simplify the question to only 1 token lookahead. Given the next token, the next rule can be determined by checking the first set of the non-terminal.

2.4. LL(1) grammar

2.5. Non-LL grammars

Example 2.2.

Consider the following grammar whose language is $\{a^i b^j, i \geq j\}$:

$$[1]\ S \to \mathtt{a}S$$
$$[2]\ S \to P$$
$$[3]\ P \to \mathtt{a}P\mathtt{b}$$
$$[4]\ P \to \varepsilon.$$

The grammar itself is clearly not LL(1) because $\mathrm{FIRST}(\mathtt{a}S) \cap \mathrm{FIRST}(P) = \{a\}$.

The intuition that *only upon seeing b can we decide which rule to apply* is the key to understanding the non-LL nature of this language. Actually, we can prove that no LL(k) parser exists for this language.

...

2.6. PEG and Packrat Parsing

Definition 2.4. Parsing Expression Grammar (PEG)

Failure-driven parsing?

- **Terminal** a: recognizes the string a.
- $e_1 e_2$: first recognizes e_1. If successful, it recognizes e_2. If one of them fails, the whole expression fails.
- e_1 / e_2: first recognizes e_1. If e_1 fails, it recognizes e_2 from the same position.
  - $S \leftarrow \mathtt{i}S\mathtt{e}S / \mathtt{i}S / \mathtt{a}$ can correctly parse dangling else.
- e_1 $*$: similar to regular expression, but greedy.
  - Meaning $0 * 0$ cannot recognize any string.
- **Predicates**: lookahead the symbols, and succeed if the lookahead satisfies the predicate.
  - $\&e$: if lookahead e succeeds, then it succeeds.
  - $!e$: if lookahead e fails, then it succeeds.
  - Example: $C \leftarrow$ `open` $(C / (!$`close any`$)) *$ `close` will handle nested parentheses.

...

2.7. Bottom-Up Parsing with LR

3. Semantic Analysis

3.1. Attribute Grammar

$$\text{Attribute Grammar} \coloneqq \text{Context-Free Grammar} + \text{Attributes}.$$

To specify the semantics of a programming language, a pratical way is to use attribute grammar. The idea is to attach attributes to the non-terminals of a context-free grammar, and define the rules for computing the attributes from the involved non-terminals. That is, in the view of parse trees, attribute grammar is a way to annotate the nodes of the parse tree with values. Hence the use of attribute grammar is also called **syntax-directed definition (SDD)**.

3.2. Synthesized and Inherited Attributes

> **Definition 3.1.** Synthesized and Inherited Attributes
>
> Given a grammar $G = (V, T, P, S)$, an attribute grammar is a set of rules of the form $X.a = f(X_1.a_1, ..., X_n.a_n)$, where X is a non-terminal, a is an attribute, and f is a function that computes the value of $X.a$ from the values of $X_1.a_1, ..., X_n.a_n$.
>
> - **Synthesized attributes**: the value of $X.a$ is computed from the values of the attributes of the children of X. Information flows from the leaves to the root, a *synthesis* process.
> - **Inherited attributes**: the value of $X.a$ is computed from the values of the attributes of the parent of X, and also its siblings N_i. Information flows from the root to the leaves, a *propagation* process.

According to the definition, we can see that:
- Synthesized attributes often appear in the form where the attributes of LHS is computed from the attributes of RHS; and vice versa for inherited attributes.

3.3. Dependency Graph

To actually compute the attributes, we can build a dependency graph to represent the dependencies between attributes, and thus a topological sort can be used to compute the attributes in the right order.

> **Definition 3.2.** Dependency Graph
>
> Given an attribute grammar, the dependency graph is a directed graph where each node is an attribute, and there is an edge from $X.a$ to $Y.b$ if the computation of $Y.b$ depends on the value of $X.a$.

However, there might be cycles in the dependency graph. Sometimes this is acceptable: e.g. modelling the evaluation of a loop with attributes, where we can properly select a starting point, repeatedly update the attributes whenever the attributes that it depends on are updated, and terminate when the attributes reach a fixed point.

But in general, we wish no cycles in the dependency graph. This goal isn't always easy to justify, so we switch our focus onto some specific types of attribute grammars where we can guarantee the acyclicity of the dependency graph. And most importantly, we can compute the attributes in a single pass, combined with the parsing process.

3.4. S-attributed and L-attributed Grammars

4. Target Code Generation

4.1. Intra-Block Generation

4.1.1. `getReg`

- **Goal**: reduce the number of memory accesses by using registers.
- **How?**: take `x = y op z` as an example. Our aim is to select registers for the result `x` and the operands `y` and `z`.
  - For loading operand `y`:
    - If `y` is in register, we can use it directly.
    - If there is a register available, we can use it to store the result.

  - Otherwise suppose R is a candidate that stores the value of another variable v.
    - If the address descriptor of v contains other positions, we can directly use v to store the result.
    - If v is x, we can use v as long as z is not x, so that operand y will not overwrite z.
    - Otherwise, we need to spill v to memory, and load y into the register.
- For storing result x:
  - If x is in register, and the register is not used by other variables, we can use it directly.
  - If a operand y is not active after this instruction, we can use y to store the result as long as the register is not used by other variables.
  - Otherwise, a spill is needed.
- For a copy x = y, just get the register of y and let $R_x = R_y$.

4.2. Register Allocation